Lavoro parallelo sul fisso

Ragionamento, pseudocodice e strumenti Unreal

Ramo `claude/project-thread-v1plhd`

25 ottobre 2026

Indice

1 Cosa c'è in questo documento **1**

2 Muri: step up e step down separati **1**
2.1 Ragionamento 1
2.2 Pseudocodice 2
2.3 Strumenti Unreal 2
2.4 Test 2
2.5 Costo delle query 2

3 Unione del ramo M4 **2**

4 Editor dei combattimenti (M5): i passi che aspettavano M4 **2**
4.1 E3: fasi derivate del colpo (`f100773`) 2
4.2 E11: regole di validazione M4, "Validate all" e pannello (`5a31adc`) 3
4.3 E5b: righe Hitbox e HurtboxModifier nella timeline (`281fa38`) 4
4.4 E12: riga dei dati del colpo e anteprima del knockback (`58af658`) 4

5 Cosa controllare a mano stasera **5**

1 Cosa c'è in questo documento

Il lavoro fatto sul fisso mentre il portatile porta avanti M4: lo step up/step down dei muri, l'unione del ramo M4 e i passi dell'editor dei combattimenti (M5) che aspettavano i dati di M4. Per ogni parte: perché è fatta così, lo pseudocodice della logica principale, le classi e gli strumenti di Unreal usati, i test e cosa resta da controllare a mano nell'editor (sezione 5).

2 Muri: step up e step down separati

2.1 Ragionamento

Il look-ahead del wall slide v2 guarda la parete davanti al personaggio e classifica ciò che trova (`EGSTN_WallEdgeKind`): *Bump* (continuo, si scavalca), *Slope*, *Concave* (angolo interno), *Convex* (spigolo esterno), *Blocked*. Prima c'era un solo limite, `MaxStepHeight`, usato sia per ciò che sporge dal muro sia per ciò che rientra. Come nella modalità di camminata, ora i limiti sono due: **step up** (`MaxStepHeight`, mostrato come "Max Step Up Height") e **step down** (`MaxStepDownDepth`), entrambi 10 cm di default, quindi il comportamento non cambia finché un designer non li modifica. Il nome della proprietà esistente è rimasto lo stesso, così i valori già salvati nei Blueprint si caricano senza redirect.

Nota sulla rientranza da 15 cm: il personaggio ci entra (l'adesione lo tira verso la parete) e ne esce risalendo di 15 cm; quella risalita è uno *step up*. Per attraversarla senza rallentare va alzato lo step up a 15 cm o più.

2.2 Pseudocodice

```
ClassifyAhead(Offset, Angolo, ...):
  // Offset < 0: la superficie sporge verso il personaggio; > 0: rientra
  Limite = (Offset <= 0) ? MaxStepHeight : MaxStepDownDepth
  se |Offset| <= Limite:
      se l'angolo non èdolce -> Bump (si scavalca, mai angolo)
      se |Offset| quasi zero -> None
      se l'offset èquello di un pendio -> Slope, altrimenti Bump
  altrimenti se angolo dolce -> Slope
  altrimenti se angolo oltre il limite -> Blocked
  altrimenti -> Offset > 0 ? Convex : Concave
```

2.3 Strumenti Unreal

`USTRUCT FGSTN_WallEdgeSettings` con `UPROPERTY(EditAnywhere, meta=(DisplayName, ClampMin, ForceUnits="cm"))`; la funzione pura in `UGSTN_WallEdgeUtils` (`UBlueprintFunctionLibrary`); test con `IMPLEMENT_SIMPLE_AUTOMATION_TEST`.

2.4 Test

`GSTN.Wall.Edge.StepUpAndDownAreSeparate` (classificazione con limiti diversi) e `GSTN.Wall.V2.Lane.RecessWithinTheStepUpIsNotACorner` (la rientranza da 15 cm, con i limiti a 20 cm, si attraversa a velocità piena). 74 test dei muri verdi. Commit `8be7b4d`.

2.5 Costo delle query

Il campionamento che distingue una curva da uno spigolo porta il v2 a 5–5,7 query per tick (massimo 9) sulle corsie con curve e angoli, contro le 3,5 della parete piatta. Accettato per ora; se il wall slide comparirà in un profilo, i primi interventi sono campionare la catena un tick sì e uno no e tenerla in cache finché lo stesso angolo resta nel look-ahead (`wall-slide-v2.md` §6.13).

3 Unione del ramo M4

Il ramo del portatile (`feature/gas-npp-5.8`, M4 fino a H13b) è stato unito nel ramo del fisso (`5d0b7da`). Un solo conflitto, l'elenco dei tag di combattimento in `GSTN_CombatTags.cpp`: tenuti i tag di entrambi i lati. `Plugins/GSTNSurface` è identico sui due rami. 806 test verdi dopo l'unione.

4 Editor dei combattimenti (M5): i passi che aspettavano M4

Quattro passi del piano `m5-editor-plan.md`, uno per commit, tutti sul plugin `GSTNCombatEditor` tranne E3 (che vive nel Core di combattimento). Test: da 806 a 846, tutti verdi. Le deviazioni dal piano sono in `docs/notes/m5-doc-deltas.md`.

4.1 E3: fasi derivate del colpo (`f100773`)

Ragionamento. Startup, attivo e recovery di una mossa non si scrivono a mano: si ricavano dalle tracce hitbox della timeline, così il dato mostrato non può divergere da ciò che il gioco esegue.

`FGSTN_TimelinePhases` sta nel Core (`GSTNCombatCore`, `Timeline/GSTN_TimelinePhases.h`) perché è logica pura, utile anche fuori dall'editor; la barra delle fasi dell'editor (`FGSTN_PhaseBarData`) resta solo un modello di disegno e legge questi numeri senza ricalcolarli. Una traccia hitbox senza specifiche conta come attiva (è un guscio creato da un autore: lo segnala il validatore); una traccia le cui varianti ignorano tutte e quattro le relazioni non conta.

```
finestre = [(max(inizio,0), fine) per ogni traccia Hitbox con fine > inizio e
    ContaComeAttiva(traccia)]
ordina per inizio; unisci se prossima.inizio <= ultima.fine
Attivo = prima.inizio .. ultima.fine ; Startup = prima.inizio ; PrimoColpo (1-based) =
    prima.inizio + 1
Recovery = max(FAF - FineAttivo, 0) ; nessuna finestra -> bHasActive = false
```

Unreal. `USTRUCT` nel Core, `FText` per il riassunto (`FormatSummary(TickRate)`), array riservato una volta sola (nessuna allocazione per mosse senza hitbox). **Test**: `GSTNCombat.Core.TimelinePhases.*` (10).

Da notare. A 62,5 Hz 30 frame sono 480 ms; l'esempio della specifica diceva 484 (calcolato a 62 Hz).

4.2 E11: regole di validazione M4, "Validate all" e pannello (`5a31adc`)

Ragionamento. Stesso schema di E10: funzioni pure che riempiono un report, chiamate sia dalla Data Validation di Unreal sia dal pannello, così pannello e salvataggio dicono sempre la stessa cosa (`UGSTN_CombatValidatorBase::BuildReportForAsset`). Dove il Core avvisa già (socket non baked, lunghezza del bake, Armor senza soglia) il suo messaggio viene classificato e non duplicato.

```
HitboxKeys: frame della chiave < 0 o >= lunghezza finestra -> errore
  stesso HitGroup, finestre sovrapposte, stessa Priority, Data diversi, forma diversa ->
      avviso
HitData: danno/knockback/crescita/moltiplicatori/stagger/hitstop negativi -> errore
  FixedKnockback >= 0 insieme a crescita -> avviso; HitType o Cue non registrati ->
      errore
  le varianti che sovrascrivono campi sono controllate allo stesso modo
Varianti: chiave fuori da Combat.Relationship, ShapeScale <= 0 senza ignore -> errore
HurtState: Armor con soglia <= 0 e non infinita -> errore
BakedSocketMissing / BakedLength -> errore ; BakedStale -> info
Hurtbox set: vuoto, nomi doppi, estensioni, tag fuori da Combat.Hurtbox -> errore
```

Pannello. `GSTNValidateAll` raccoglie gli asset (salta `/Engine` e `/Temp`) e trova i Blueprint di abilità tramite `GetDerivedClassNames` e il tag `GeneratedClass` dell'Asset Registry, *senza caricarli*. Il modello `FGSTN_ValidationPanelModel` non ha widget (testabile da solo); `SGSTN_ValidationPanel` è un `SCompoundWidget` con `STreeView`. Doppio clic: apre l'asset e porta la timeline sul frame e sulla chiave del problema. La scheda Frame Data rivalida 250 ms dopo l'ultima modifica (`FTSTicker`) e le corsie mostrano una linea rossa o gialla. Registrazione: scheda nomade nel gruppo "GSTN Combat", comando `gstn.Combat.ValidateAll`, pagina del Message Log.

Test. `GSTNCombat.Editor.Validate.*` (14) e le istantanee `GSTN.Visual.Validation.Panel`, `.LaneMarkers`.

Rimandato. "Bake all" e la parte hash di `BakedStale` (servono il baker di E8); controlli su scheletro e ossa (servono la mesh dell'hurtbox set, E9); la fascia "clank" della Priority (è di M6).

4.3 E5b: righe Hitbox e HurtboxModifier nella timeline (`281fa38`)

Ragionamento. Una riga per traccia con una sotto-corsia per ogni hitbox, invece di una riga per hitbox: la matematica di righe, selezione e trascinamento del modello E2/E4 assume righe da 24 px, e cambiarla avrebbe toccato tutto il modello. Le chiavi restano relative all'inizio della traccia (`FGSTN_HitboxKey::Frame`), quindi spostare la finestra sposta le chiavi.

```
AddKey su un percorso: chiave = UGSTN_HitShapeUtils::EvaluateKeys(chiavi della corsia,
    frame relativo)
   // la stessa funzione del runtime: aggiungere una chiave non cambia il percorso
AddKey senza corsia: corsia della prima chiave selezionata sulla traccia, altrimenti
    corsia 0
Duplica / incolla: le copie restano nella corsia d'origine, in ordine di corsia
    crescente
Nuova traccia Hitbox: una sfera e una chiave; parte vuota di proposito, quindi nasce
    come errore
```

Unreal. Interfaccia dell'adapter estesa con `GetLaneCount`, `GetKeyLane`, `AppendKeyInLane` (con default), colore per stato tramite il nuovo virtuale `GetTrackColor`. **Test**: 8 sul modello e sulla registry, istantanea `GSTN.Visual.Timeline.HitboxRows`. **Limiti**: con cinque o più hitbox la riga è affollata (il pannello Details ha la lista completa); una chiave trascinata può uscire dalla finestra (lo segnala `HitboxKeys`).

4.4 E12: riga dei dati del colpo e anteprima del knockback (`58af658`)

Ragionamento. `FGSTN_HitDataCustomization` (`IPropertyTypeCustomization` per `FGSTN_HitData`) raggruppa i campi (danno, knockback, hitstun, hitstop, stagger, colpo, impatto, estensioni, anteprima) con un riassunto su una riga e una frase che spiega Special Angle e Yaw Source. Le estensioni usano l'editor di array standard, così la customization di un altro plugin compare dentro. Il quadrante dell'angolo (`SGSTN_AngleDial`) è un solo passo di undo: transazione aperta alla pressione, scritture interattive, chiusura al rilascio; Esc ripristina.

Anteprima. Non reimplementa la formula: costruisce lo stesso input del passo runtime `UGSTN_HitStep_ComputeKnockback` e chiama il modello delle regole di default, quindi non può dare numeri diversi dal gioco.

```
input = {colpo, su=+Z, peso, percentuale, salute 1, soglia stagger = regole.
    StaggerThreshold}
se Sakurai: K = modello(input con direzione nulla).Knockback
input.Direzione = LaunchDirection(yaw, angolo, angolo speciale, su, avanti=+X, a terra,
    K, regole.Sakurai)
risultato = modello.Compute(input) // knockback, tier, stagger, hitstun, hitstop, angolo
     effettivo
```

Unreal. `IPropertyTypeCustomization`, `SGameplayTagCombo` filtrato su `Combat.HitType` (nuova dipendenza dal plugin `GameplayTagsEditor`), transazioni dell'editor e `IPropertyHandle::SetValue` per il quadrante. **Test**: `GSTNCombat.Editor.HitData.*` (4, compreso il caso a mano $K = (400 + 10 \cdot 6) \cdot 100/50 = 920$) e l'istantanea `GSTN.Visual.HitData.Row`. **Limiti**: la vittima di prova (peso, percentuale) vive nella riga ed è solo di sessione; l'anteprima ignora scale delle hurtbox, DI, tumble e yaw diversi dall'orientamento dell'attaccante; usa solo le regole di default.

5 Cosa controllare a mano stasera

1. **Muri**: in `SandboxCharacter_Mover`, modalità WallSlidingV2, sezione *Edges*: ci sono "Max Step Up Height" e "Max Step Down Depth". Sulla mappa `L_WallSlideAB`, corsia delle rientranze: con lo step up a 15–20 cm la rientranza da 15 cm non deve più rallentare.

2. **Timeline, righe hitbox**: apri un asset timeline, aggiungi una traccia Hitbox con doppio clic, trascina le chiavi tra le sotto-corsie, premi K con una chiave selezionata, Ctrl+trascina per copiare, poi annulla e ripeti.

3. **Dati del colpo**: seleziona un hitbox; nel Details trascina il quadrante dell'angolo (un solo undo, Esc ripristina), cambia il tag HitType, aggiungi un'estensione, leggi la riga di anteprima del knockback.

4. **Validazione**: crea un errore apposta (danno negativo o chiave oltre la finestra); entro circa 250 ms reagiscono la scheda Frame Data e la linea sulla corsia. Premi F7 e controlla il Message Log.

5. **Validate all**: Window, GSTN Combat, Validation, "Validate all", poi doppio clic su un problema di un asset timeline e di una timeline inline in un Blueprint di abilità: playhead e selezione devono finire sul frame giusto.